CHAPITRE 3

Structures de données

PTSI

2026 – 2027

PLAN DU CHAPITRE

3.1 Listes 2

3.2 Tuples 9

3.3 Chaînes de caractères 10

3.4 Dictionnaires 13

3.1 Listes

Définition 3.1 · Liste

Une *liste* est une collection finie et ordonnée de valeurs, éventuellement de types différents. En Python, elle se note entre crochets, ses éléments étant séparés par des virgules.

EXEMPLE 3.2

`[True, 4, 5, 3.0]` est une liste.

Construire une liste

Une liste peut être obtenue de plusieurs façons :

- en donnant explicitement ses éléments entre crochets, comme ci-dessus ;
- par **concaténation** de deux listes, à l'aide de `+` ;
- avec `list(iterable)`, qui construit une liste à partir de n'importe quel objet itérable (un `range`, une chaîne de caractères, …) ;
- par **compréhension**, selon le schéma `[f(x) for x in iterable if P(x)]`, où `f(x)` est une expression dépendant de `x` et `P(x)` une condition booléenne facultative ;
- par *slicing* (tranchage), qu'on détaille un peu plus loin.

console

```
In [1]: [1,2,3] + [4,5,6]
Out[1]: [1, 2, 3, 4, 5, 6]

In [2]: list(range(4))
Out[2]: [0, 1, 2, 3]

In [3]: list('truc')
Out[3]: ['t', 'r', 'u', 'c']

In [4]: [x*x for x in range(6) if x%2==1]
Out[4]: [1, 9, 25]
```

Accéder à un élément

Définition 3.3 · Longueur et indices

Pour une liste `L`, sa *longueur* (le nombre de ses éléments) s'obtient avec `len(L)`. Si n désigne cette longueur, ses éléments sont indexés par les entiers de 0 à $n-1$.

console

```
In [1]: L = list(range(2,8))    # entiers de 2 a 7
In [2]: L[3]
Out[2]: 5
In [3]: L[len(L)-1]
Out[3]: 7
```

Un indice **négatif** i, compris entre -1 et $-n$, est quant à lui interprété comme $n+i$, ce qui donne

notamment un accès direct au dernier élément :

console

```
In [4]: L[-1]    # dernier element
Out[4]: 7
In [5]: L[-6]    # premier element (n=6 ici)
Out[5]: 2
```

Tout autre indice produit une erreur, qu'il faudra apprendre à reconnaître :

console

```
In [6]: L[len(L)]
IndexError: list index out of range
```

Parcourir une liste

La boucle `for` permet également de faire parcourir à une variable, un par un, tous les éléments d'une liste :

console

```
In [1]: L = list(range(2,8))    # entiers de 2 a 7
In [2]: for i in L:
   ...:     print(i)
   ...:
2
3
4
5
6
7
```

Pour parcourir la liste en sens inverse, il suffit d'envelopper celle-ci dans la fonction `reversed` :

console

```
In [3]: for i in reversed(L):
   ...:     print(i)
   ...:
7
6
5
4
3
2
```

Modifier un élément

On modifie l'élément d'indice *i* d'une liste `L` exactement comme on affecterait une variable, avec les mêmes règles sur *i* que ci-dessus :

console

```
In [7]: L[2] = 100
In [8]: L
Out[8]: [2, 3, 100, 5, 6, 7]
```

Slicing (tranchage)

Définition 3.4 · Tranchage

Pour une liste `L`, l'extraction `L[d:f]` produit une **nouvelle** liste formée des éléments `L[d]`, `L[d+1]`, …, `L[f-1]` (d inclus, f exclu). Omettre d revient à prendre $d = 0$; omettre f revient à prendre $f = n$ (la longueur de `L`). Ce mécanisme tolère les indices hors bornes, et si $d \geqslant f$, le résultat est la liste vide.

console

```
In [9]: L
Out[9]: [2, 3, 100, 5, 6, 7]
In [10]: L[2:5]
Out[10]: [100, 5, 6]
In [11]: L[5:2]
Out[11]: []
In [12]: L[:3]
Out[12]: [2, 3, 100]
In [13]: L[:10]
Out[13]: [2, 3, 100, 5, 6, 7]
```

On peut également préciser un **pas**, positif ou négatif – l'interprétation est la même que pour `range` :

console

```
In [14]: L[::2]
Out[14]: [2, 100, 6]
In [15]: L[::-1]    # inverse la liste
Out[15]: [7, 6, 5, 100, 3, 2]
```

Quelques méthodes utiles

Python étant un langage orienté objet, plusieurs *méthodes* s'appliquent aux listes, avec la syntaxe générale `objet.methode(parametres)`. On se limite essentiellement à `append` et `pop` :

méthode	**effet**
`L.append(x)`	ajoute `x` en fin de `L`
`L.extend(T)`	ajoute les éléments de `T` en fin de `L` (équivalent à `L+=T`)
`L.insert(i,x)`	insère `x` en position `i`, en décalant les suivants
`L.remove(x)`	supprime la première occurrence de `x` (erreur si absent)
`L.pop()`	supprime et renvoie le dernier élément de `L`
`L.pop(i)`	supprime et renvoie l'élément d'indice `i`
`L.index(x)`	renvoie l'indice de la première occurrence de `x`
`L.count(x)`	renvoie le nombre d'occurrences de `x` dans `L`
`L.sort()`	trie `L` par ordre croissant (en place)
`L.reverse()`	inverse l'ordre des éléments de `L` (en place)

console

```
In [1]: L0 = [12, 8, 15]
In [2]: L0
Out[2]: [12, 8, 15]

In [3]: L0.append(17)
In [4]: L0
Out[4]: [12, 8, 15, 17]

In [5]: L0.extend([9, 6])
In [6]: L0
Out[6]: [12, 8, 15, 17, 9, 6]

In [7]: L0.insert(2, 20)
In [8]: L0
Out[8]: [12, 8, 20, 15, 17, 9, 6]

In [9]: L0.remove(20)
In [10]: L0
Out[10]: [12, 8, 15, 17, 9, 6]

In [11]: L0.pop()
Out[11]: 6
In [12]: L0
Out[12]: [12, 8, 15, 17, 9]

In [13]: L0.pop(0)
Out[13]: 12
In [14]: L0
Out[14]: [8, 15, 17, 9]

In [15]: L0.index(15)
Out[15]: 1

In [16]: L0.count(9)
Out[16]: 1

In [17]: L0.sort()
In [18]: L0
Out[18]: [8, 9, 15, 17]

In [19]: L0.reverse()
In [20]: L0
Out[20]: [17, 15, 9, 8]
```

Remarque

Attention : ces méthodes ne renvoient en général rien, elles modifient l'objet directement. C'est le cas d'`append` : pour ajouter `x` en fin de `L`, on écrit `L.append(x)`, surtout pas `L=L.append(x)`. Cette dernière expression s'évalue en effet en `None`, et écrire `L=L.append(x)` reviendrait à écraser `L` par

> `None`.

Listes et références

Définition 3.5 · Référence

En Python, une variable désignant une liste ne contient pas directement ses éléments : elle contient une *référence* vers l'emplacement mémoire où la liste est réellement stockée. Affecter une liste à une autre variable (`U = T`) copie donc uniquement cette référence, pas le contenu. `T` et `U` désignent alors **la même** liste en mémoire.

console

```
In [1]: T = [10, 20, 30]
In [2]: U = T
In [3]: T[0] = 99
In [4]: T.append(40)
In [5]: print(U)
[99, 20, 30, 40]
```

Modifier `T` modifie donc également `U` ! Pour obtenir une véritable copie, indépendante en mémoire, on peut par exemple utiliser un slicing complet, `U = T[:]`, ou de façon équivalente la méthode `T.copy()` :

console

```
In [1]: T = [10, 20, 30]
In [2]: U = T[:]
In [3]: T[0] = 99
In [4]: T.append(40)
In [5]: print(U)
[10, 20, 30]
```

console

```
In [1]: T = [10, 20, 30]
In [2]: U = T.copy()
In [3]: T[0] = 99
In [4]: T.append(40)
In [5]: print(U)
[10, 20, 30]
```

Listes de listes

On rencontrera souvent des listes contenant elles-mêmes des listes, en particulier pour représenter des matrices. L'accès à leurs éléments s'enchaîne simplement :

console

```
In [1]: L = [[1,2],[3,4,5]]    # une liste de deux listes
In [2]: L[0]
Out[2]: [1, 2]
In [3]: L[1][2]
Out[3]: 5
In [4]: L[0][1] = 9
In [5]: L
Out[5]: [[1, 9], [3, 4, 5]]
```

Nouvelle mise en garde concernant les références : recopier une liste de listes avec `[:]` ne recopie que le premier niveau, pas les listes qu'elle contient.

console

```
In [1]: T = [[1,2],[3,4]]
In [2]: U = T[:]
In [3]: U[0][0] = 99
In [4]: print(T)
[[99, 2], [3, 4]]
```

Les éléments de `U` restent en effet des références vers les mêmes sous-listes que celles de `T` : il aurait fallu écrire `U = [A[:] for A in T]` pour les recopier également. On notera que ce problème resurgirait de la même façon avec une liste de listes de listes – une situation heureusement rare en pratique. Pour recopier un objet à n'importe quelle profondeur, le module `copy` fournit une fonction `deepcopy` dédiée à cet usage.

3.2 Tuples

Définition 3.6 · Tuple

Un *tuple* (ou *n*-uplet) ressemble à une liste, mais ne peut être modifié une fois créé : on ne peut ni changer un de ses éléments, ni lui en ajouter ou en retirer. On dit qu'il s'agit d'une structure *immuable* (par opposition aux listes, *mutables*).
Cette rigidité a une contrepartie avantageuse : un tuple occupe peu de mémoire et l'accès à ses éléments est rapide.

Un tuple se construit en énumérant ses éléments séparés par des virgules, éventuellement entre parenthèses : celles-ci ne sont d'ailleurs obligatoires que pour lever une ambiguïté, ou lorsque le tuple ne contient qu'un seul élément (`a,` ou `(a,)`, la virgule finale étant alors indispensable). Le tuple vide se note `()`.

Les opérations déjà vues sur les listes (concaténation, slicing, `len`, ...) s'appliquent également aux tuples :

console

```
In [1]: p = 3, False, 2.5
In [2]: q = (7,)
In [3]: r = ((1,2),)    # r est un tuple dont l'unique element est un tuple
In [4]: p+q
Out[4]: (3, False, 2.5, 7)
In [5]: p[1:]
Out[5]: (False, 2.5)
In [6]: p+r
Out[6]: (3, False, 2.5, (1, 2))
In [7]: len(p+r)
Out[7]: 4
```

En revanche, toute tentative de modification échoue :

console

```
In [8]: p[0] = 10
TypeError: 'tuple' object does not support item assignment
```

Déconstruire un tuple

On peut *déconstruire* un tuple en affectant simultanément ses éléments à plusieurs variables, réunies elles aussi en tuple :

console

```
In [1]: pair = (5, 6)
In [2]: (a,b) = pair
In [3]: print(a) ; print(b)
5
6
```

Les parenthèses autour du tuple de variables sont facultatives, si bien qu'on écrit en pratique plutôt `a,b=pair`. Si le nombre de variables ne correspond pas au nombre d'éléments du tuple, Python le

signale immédiatement :

console

```
In [4]: a,b,c = pair
ValueError: not enough values to unpack (expected 3, got 2)

In [5]: triplet = 1,2,3
In [6]: a,b = triplet
ValueError: too many values to unpack (expected 2)
```

Ce mécanisme explique au passage l'écriture, déjà rencontrée, permettant d'échanger le contenu de deux variables sans passer par une variable temporaire : `a,b=b,a` construit d'abord le tuple `(b,a)`, avant de le déconstruire immédiatement sur `a` et `b`.

3.3 Chaînes de caractères

Définition 3.7 · Chaîne de caractères

Une *chaîne de caractères* est une suite finie de caractères quelconques. En Python, elle s'écrit en encadrant les caractères voulus par des apostrophes ou par des guillemets : `'Bonjour'` et `"Bonjour"` désignent ainsi exactement la même chaîne. Comme les tuples, les chaînes sont **immuables** : impossible d'en modifier un caractère une fois créées.

Concaténation, accès, slicing

Les chaînes se comportent, de ce point de vue, comme les tuples : `+` les concatène, `len` donne leur longueur, et l'accès à un caractère (ou à une tranche) suit exactement les mêmes règles d'indexation que pour les listes :

console

```
In [1]: "Le chat" + " mange."
Out[1]: 'Le chat mange.'

In [2]: a = "Un langage clair"
In [3]: a[0] ; a[5] ; a[-1]
Out[3]: 'U'
Out[3]: 'n'
Out[3]: 'r'

In [4]: a[::2]
Out[4]: 'U agg li'
```

L'immutabilité se manifeste par une erreur explicite dès qu'on tente une modification directe ; il faut alors reconstruire une nouvelle chaîne, qu'on peut éventuellement réaffecter à la même variable :

console

```
In [5]: b = 'jardin'
In [6]: b[0] = 'h'
TypeError: 'str' object does not support item assignment

In [7]: b = 'h' + b[1:]
In [8]: b
Out[8]: 'hardin'
```

Quelques méthodes utiles

Comme les listes, les chaînes disposent de méthodes – non exigibles, mais qui peuvent faire l'objet de questions si leur usage est rappelé. Ces méthodes ne modifient jamais la chaîne (immuable, rappelons-le), mais en renvoient une nouvelle :

méthode	**effet**
`s.split(c)`	sépare `s` autour du caractère `c`, en une liste de chaînes
`s.join(L)`	opération inverse : assemble les chaînes de la liste `L` en les séparant par `s`
`s.count(c)`	compte le nombre d'occurrences de `c` dans `s`
`s.find(c)`	renvoie la première position de `c` dans `s`, ou -1 si absent
`s.rjust(n,c)` / `s.ljust(n,c)`	complète `s` à droite / à gauche avec le caractère `c` (l'espace par défaut) jusqu'à atteindre n caractères
`s.replace(s1,s2)`	remplace toutes les occurrences de `s1` par `s2` dans `s`

console

```
In [1]: 'il fait beau'.split(' ')
Out[1]: ['il', 'fait', 'beau']

In [2]: ','.join(['x','yz','w'])
Out[2]: 'x,yz,w'

In [3]: 'anticonstitutionnel'.count('n')
Out[3]: 4

In [4]: 'anticonstitutionnel'.find('t')
Out[4]: 2

In [5]: 'chat'.rjust(8,'*')
Out[5]: '****chat'

In [6]: 'chat'.ljust(8,'*')
Out[6]: 'chat****'

In [7]: 'la souris mange le fromage'.replace('mange','devore')
Out[7]: 'la souris devore le fromage'
```

Cas particuliers d'écriture

Si la chaîne doit elle-même contenir une apostrophe ou un guillemet, il suffit de l'encadrer avec l'autre délimiteur : `'"Oui", dit-il'` ou `"C'est exact"`. On peut aussi *échapper* le délimiteur gênant à l'aide d'un antislash `\` : `'Il ajouta : "c\'est exact !"'`. Enfin, l'encadrement par triples guillemets ou triples apostrophes permet d'écrire une chaîne sur plusieurs lignes, sans avoir à se soucier des apostrophes ou guillemets qu'elle contient :

console

```
In [1]: s = """Cette phrase s'etend
   ...: sur
   ...:
   ...: plusieurs lignes. C'est "long"."""
In [2]: print(s)
Cette phrase s'etend
sur

plusieurs lignes. C'est "long".
```

On retrouve dans cette chaîne les caractères spéciaux `\n` (saut de ligne) et `\t` (tabulation). L'antislash étant lui-même un caractère spécial, il doit être doublé s'il doit apparaître littéralement dans la chaîne :

console

```
In [3]: s = "\\\\t est une tabulation, \\\\n est un saut de ligne"
In [4]: print(s)
\t est une tabulation, \n est un saut de ligne
```

On peut par ailleurs se servir d'une chaîne entre triples guillemets comme d'un **commentaire multi-lignes** : dépourvue d'effet de bord (elle n'est affectée à rien), elle est simplement évaluée puis ignorée par l'interpréteur, un peu comme le serait l'expression `10` ou `2+3` tapée seule.

Conversion de types

Définition 3.8 · Conversion

Les fonctions `int`, `float` et `str` permettent de convertir une donnée d'un type vers un autre.

Une chaîne comme `"42"` reste une chaîne de caractères, pas un entier : l'expression `"42"+5` n'a ainsi *aucun sens* pour Python, qui refuse d'additionner une chaîne et un entier :

console

```
In [1]: "42" + 5
TypeError: can only concatenate str (not "int") to str

In [2]: int("42") + 5
Out[2]: 47

In [3]: str(5) + "!"
Out[3]: '5!'
```

3.4 Dictionnaires

Définition 3.9 · Dictionnaire

Un *dictionnaire* est un objet stockant des couples (*clé*, *valeur*), chaque clé ne pouvant apparaître qu'une seule fois. Un dictionnaire de langue en est un exemple typique : les clés sont les mots, les valeurs leurs définitions.

Un dictionnaire s'écrit entre accolades, chaque couple étant noté `clé:valeur`, les couples étant séparés par des virgules, sans aucune contrainte d'homogénéité, ni sur les clés, ni sur les valeurs :

EXEMPLE 3.10

```
console
In [1]: D = {'un': 1, 3: 'trois', (0,1): 'origine'}
In [2]: D
Out[2]: {'un': 1, 3: 'trois', (0, 1): 'origine'}
```

EXEMPLE 3.11

Un répertoire téléphonique est un dictionnaire. Les couples (nom,prénom) sont les clés, les numéros de téléphone sont les valeurs.

Manipuler un dictionnaire

On part le plus souvent d'un dictionnaire vide, qu'on complète ensuite couple par couple. Un dictionnaire vide s'obtient avec `dict()`, ou simplement `{}` :

```
console
In [1]: dict(), {}
Out[1]: ({}, {})
```

L'accès à la valeur associée à une clé `k` se fait via `D[k]` :

```
console
In [2]: D['un'], D[(0,1)]
Out[2]: (1, 'origine')
```

Si la clé demandée n'existe pas, Python le signale par une erreur :

```
console
In [3]: D['deux']
KeyError: 'deux'
```

On teste donc plutôt la présence d'une clé avec l'opérateur `in` avant d'y accéder :

```
console
In [4]: 'deux' in D, 'un' in D
Out[4]: (False, True)
```

Ajouter un nouveau couple, ou modifier la valeur associée à une clé existante, se fait exactement de la même façon, via `D[k] = valeur` :

console

```
In [5]: D[3] = 100           # modification d'une valeur existante
In [6]: D['nouveau'] = 'valeur'    # ajout d'un nouveau couple
In [7]: D
Out[7]: {'un': 1, 3: 100, (0, 1): 'origine', 'nouveau': 'valeur'}
```

Remarque

Attention : une clé doit obligatoirement être un objet *immuable* (entier, flottant, chaîne, tuple d'éléments immuables...). Une liste, par exemple, ne peut pas servir de clé.

Quelques méthodes utiles

méthode	effet
`len(D)`	renvoie le nombre de couples stockés dans `D`
`D.pop(k)`	supprime le couple de clé `k` de `D` (et renvoie sa valeur)
`D.keys()`	renvoie l'ensemble des clés de `D` ; on peut écrire `for k in D.keys():` ou, de façon équivalente, simplement `for k in D:`
`D.copy()`	renvoie une copie de `D`

console

```
In [8]: len(D)
Out[8]: 4

In [9]: D.pop('un')
Out[9]: 1

In [10]: D
Out[10]: {3: 100, (0, 1): 'origine', 'nouveau': 'valeur'}

In [11]: list(D.keys())
Out[11]: [3, (0, 1), 'nouveau']

In [12]: D2 = D.copy()
In [13]: D2
Out[13]: {3: 100, (0, 1): 'origine', 'nouveau': 'valeur'}
```